PROGINEER · BACKEND TECHNICAL TASK 5

Donation Management API

A RESTful CRUD API in ASP.NET Core that manages donations and never exposes the name of an anonymous donor.

Izzaldeen | ASP.NET Core · .NET 10 | EF Core 10 | SQL Server | October 2026

42/42

Postman tests passed

SUMMARY

What I built, at a glance

A backend-only Web API with full CRUD, a health check, Swagger documentation, a SQL Server database created by EF Core migrations, and an automated Postman test suite.

6
API endpoints
(5 CRUD + health)

18
Postman requests
(11 normal + 7 error cases)

42
Automated tests
all passed, 0 failed

0
Ways an anonymous
name can leak

Privacy first

Anonymous donors are shown as **"Anonymous"** in every response. Masking happens in one function that every endpoint goes through.

Correct HTTP

Standard status codes (200, 201, 204, 400, 404, 503) and RFC 7807 `ProblemDetails` error bodies.

Validated at two layers

DTO rules give clear 400 errors; a database `CHECK` constraint protects the data itself.

ARCHITECTURE

Clean layers, no over-engineering

Each layer has one job. The controller speaks HTTP, the service holds the logic, EF Core talks to the database.

Client
Swagger UI or Postman sends JSON over HTTP

→

Controllers
Model validation, maps results to status codes

→

DonationService
Business logic + the only entity → DTO mapping

→

AppDbContext
EF Core: table design, constraints, index

→

SQL Server
Database `DonationDb`, table `Donations`

```
DonationApi/
├── Controllers/   DonationsController · HealthController
├── DTOs/          Create / Update request · DonationResponse
├── Models/        Donation (entity)
├── Services/      IDonationService · DonationService
├── Data/          AppDbContext
├── Migrations/    InitialCreate
├── Postman/       collection (v2.1)
└── Program.cs     DI · ProblemDetails · Swagger
```

Why this shape

- **DTOs, not the entity:** clients cannot set `Id` or `CreatedAt`, and the response can hide the name.
- **Service layer:** keeps controllers thin and puts the privacy rule in one place.
- **No repository layer:** `DbContext` already is one; adding another would be over-engineering.

CRITICAL REQUIREMENT

How anonymous names are protected

The real name is stored in the database, but every response is built by one function, which swaps it for "Anonymous".

```
// DonationService.cs: the ONLY entity → response mapping
private static DonationResponse ToResponse(Donation d) =>
new()
{
    Id          = d.Id,
    Amount      = d.Amount,
    DonorName   = d.IsAnonymous ? "Anonymous" : d.DonorName,
    IsAnonymous = d.IsAnonymous,
    CreatedAt   = d.CreatedAt
};
```

GET all, GET by id, POST and PUT all return through `ToResponse`. A future endpoint that uses it is protected automatically.

Request POST /api/donations

```
{ "amount": 120.5, "donorName": "Secret Donor",
"isAnonymous": true }
```

Response 201 Created

```
{
  "id": 6,
  "amount": 120.5,
  "donorName": "Anonymous",
  "isAnonymous": true,
  "createdAt": "2026-10-25T06:42:40.97Z"
}
```

Postman also checks that the text "Secret Donor" appears nowhere in the response body, in the list and after updates.

DATABASE ARCHITECTURE

Table design: `Donations`

Created with EF Core Code-First and the `InitialCreate` migration on local SQL Server Express.

Column	Type	Constraint	Why
Id	`int` IDENTITY	Primary key	Auto-generated unique id
Amount	`decimal(18,2)`	NOT NULL · `CHECK ([Amount] > 0)`	Exact type for money; the database itself rejects zero or negative amounts
DonorName	`nvarchar(100)`	NOT NULL	Unicode so Arabic names are stored correctly; always stored, masked only in responses
IsAnonymous	`bit`	DEFAULT 0	Donations are public unless marked anonymous
CreatedAt	`datetime2`	DEFAULT `SYSUTCDATETIME()`	Set by the server, stored and returned in UTC (ends with `Z`)
Index	`IX_Donations_CreatedAt`		Fast "newest first" ordering for `GET /api/donations`

Defence in depth: `[Range]` on the DTO returns a clear 400 to the client, while the `CHECK` constraint keeps the data valid even if something writes to the table outside the API.

REST ENDPOINTS

Endpoints, status codes and validation

Method	Route	Status codes
GET	`/api/donations`	200
GET	`/api/donations/{id}`	200 · 400 · 404
POST	`/api/donations`	201 + Location · 400
PUT	`/api/donations/{id}`	200 · 400 · 404
DELETE	`/api/donations/{id}`	204 · 400 · 404
GET	`/api/health`	200 Healthy · 503

Validation rules

- `amount` : required, > 0 and ≤ 1,000,000
- `donorName` : required, 2–100 characters
- `isAnonymous` : required (nullable `bool?`, so "missing" ≠ false)
- `id` : must be a positive integer, `abc` → 400, not 404

Example 400 Bad Request

```
POST /api/donations
{ "donorName": "Test User" }

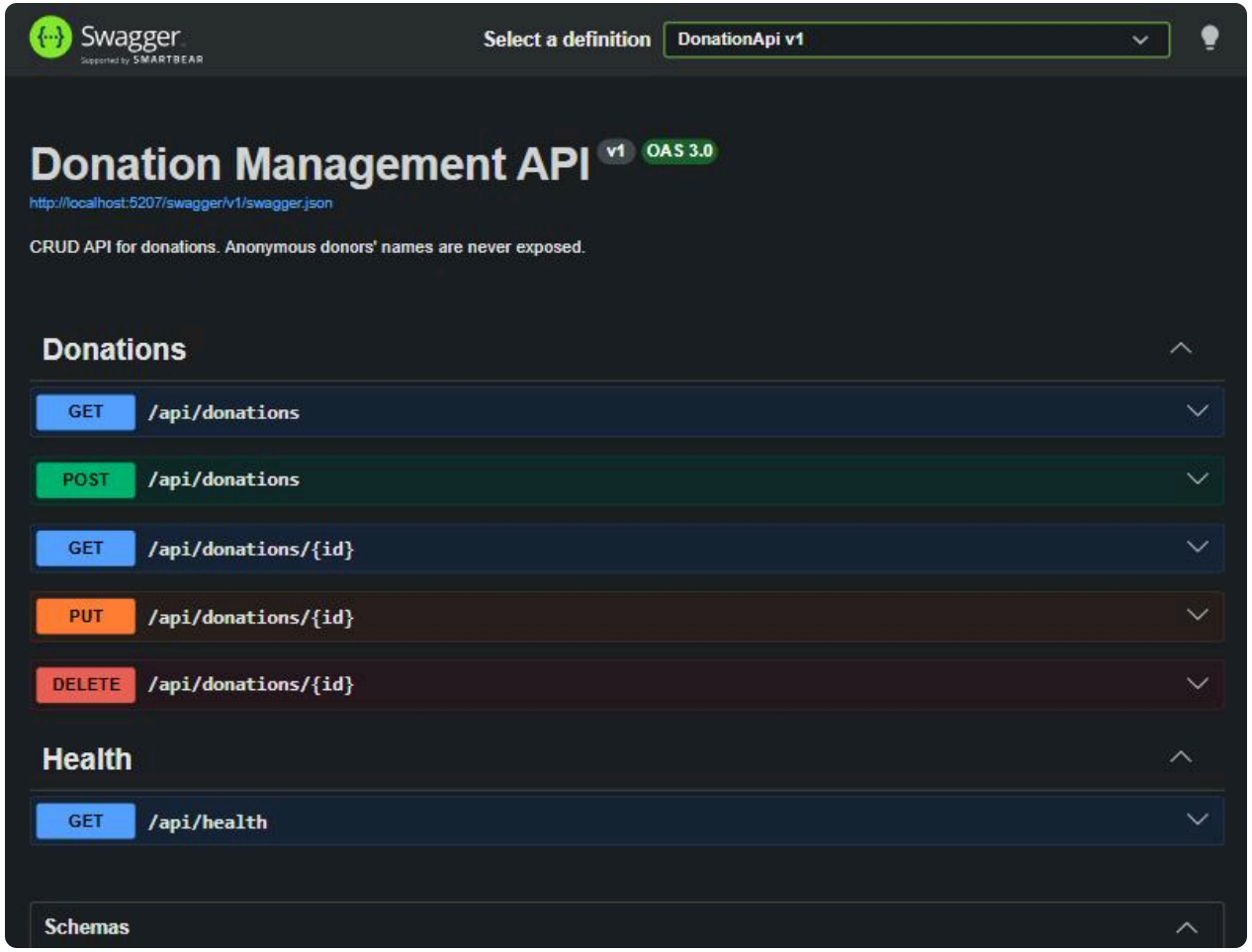
{
  "title": "One or more validation errors
occurred.",
  "status": 400,
  "errors": {
    "Amount": ["The Amount field is required."],
    "IsAnonymous": ["The IsAnonymous field is
required."]
  }
}
```

Example 404 Not Found

```
{ "title": "Donation not found", "status": 404,
  "detail": "No donation exists with id 999999." }
```

SWAGGER / OPENAPI

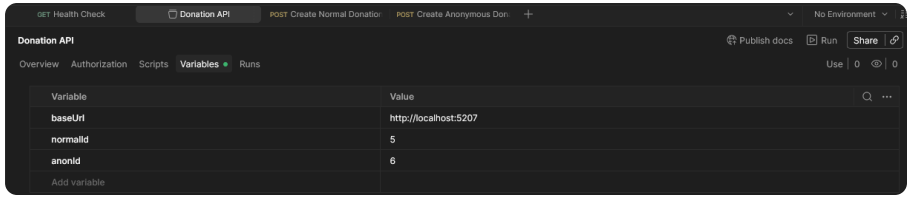
Every endpoint documented and testable



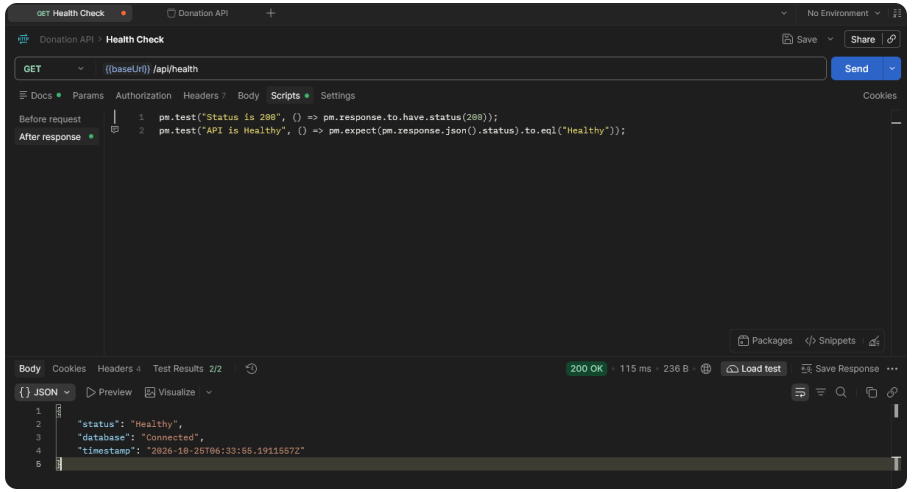
- Available at `http://localhost:5207/swagger`
- Title and description set in `AddSwaggerGen`
- Every action declares its possible responses with `[ProducesResponseType]`, so 200 / 201 / 204 / 400 / 404 / 503 all appear with their models
- Request bodies (`CreateDonationRequest`, `UpdateDonationRequest`) and `DonationResponse` listed under **Schemas**
- Each endpoint can be run with **Try it out**

From one request to a self-running test suite

1. **Collection "Donation API"** with a folder *Validation & Errors* for the negative cases.
2. **Collection variable** `baseUrl` = `http://localhost:5207`, so every URL is `{{baseUrl}}/api/...`
3. **Test scripts** in *Scripts → After response*, e.g. `pm.response.to.have.status(201)`
4. **Chained ids:** the Create requests save the new id with `pm.collectionVariables.set("anonId", ...)`; later requests use `{{anonId}}`
5. **Collection Runner** executes all 18 requests in order and reports every test



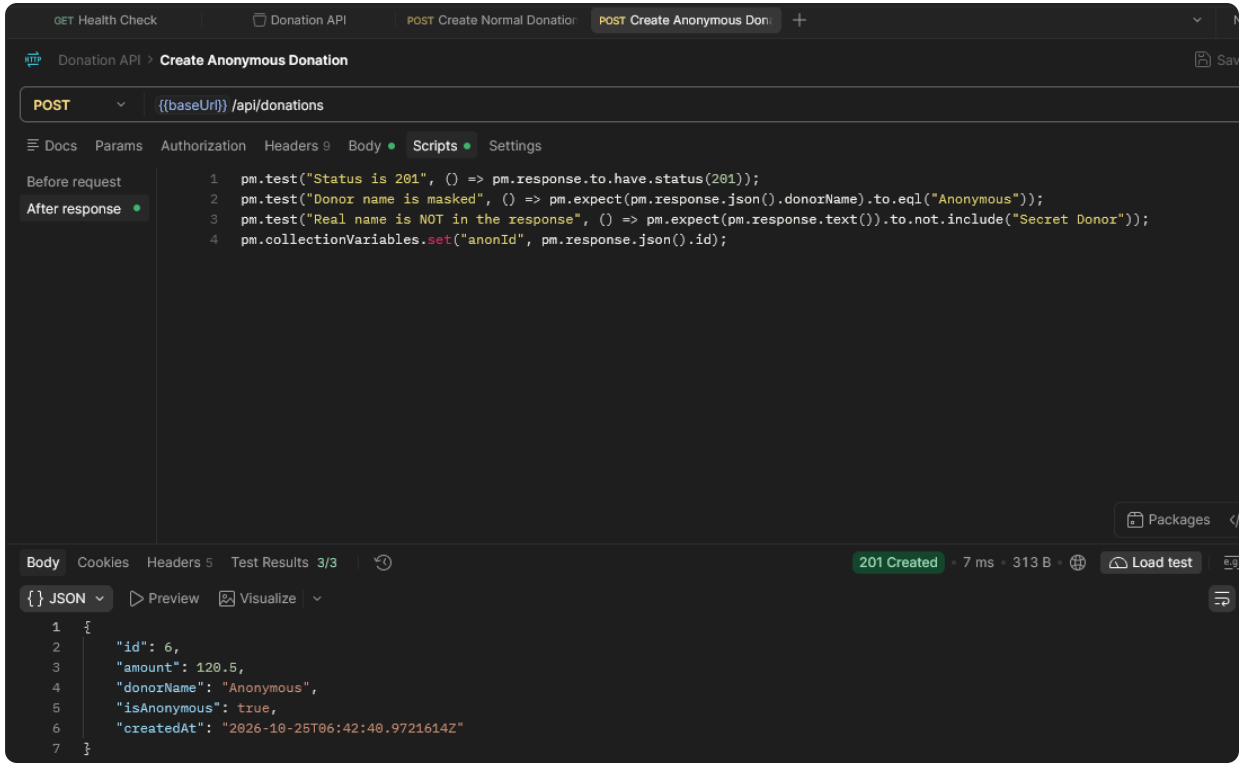
Collection variables after a run: `normalId` and `anonId` were filled automatically by the Create requests.



Health Check with its two tests: 200 OK and "Healthy" → Test Results 2/2.

POSTMAN · KEY TEST

Proving the anonymous name never leaks




```
pm.test("Status is 201", ...201);
pm.test("Donor name is masked", () =>
  expect(json.donorName).to.eql("Anonymous"));
pm.test("Real name is NOT in the response", () =>
  expect(response.text()).to.not.include("Secret
Donor"));
// save the id for the next requests
pm.collectionVariables.set("anonId", json.id);
```

The third test searches the **whole response text**, not only one field. The same check runs on *Get all*, *Get by id* and after *Update* with a new real name.

POSTMAN · COVERAGE

All required test scenarios

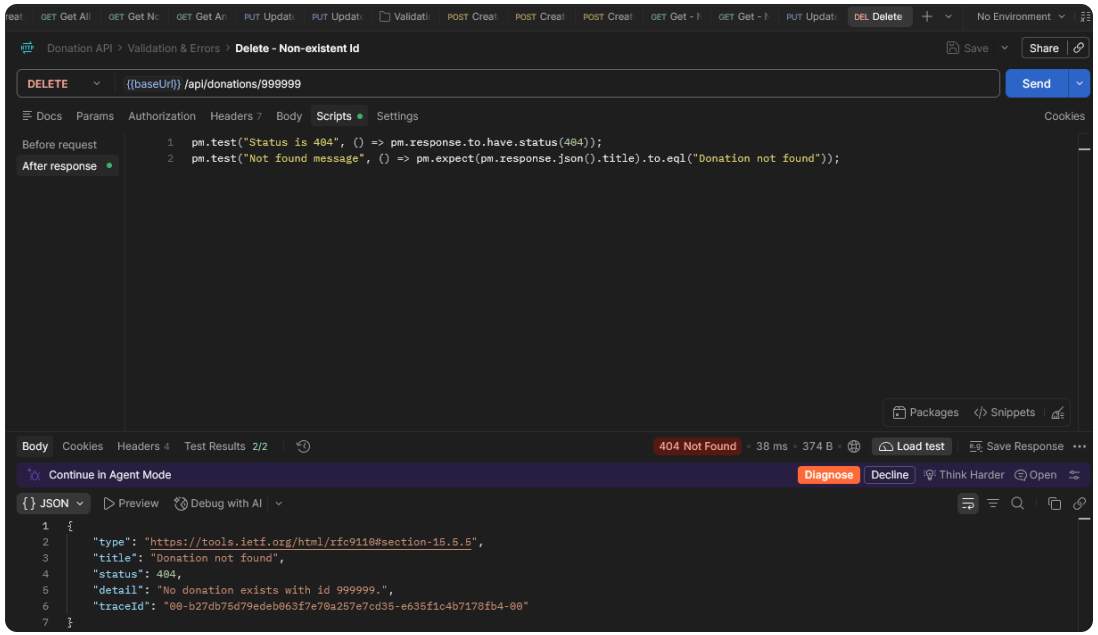
Standard scenarios 11 requests

Scenario	Expected
Health Check	200 · Healthy
Create normal / anonymous	201 · name / "Anonymous"
Get all donations	200 · anonymous masked
Get normal / anonymous by id	200 · masking verified
Update normal / anonymous	200 · still masked
Delete normal → Get deleted	204 → 404
Delete anonymous (cleanup)	204

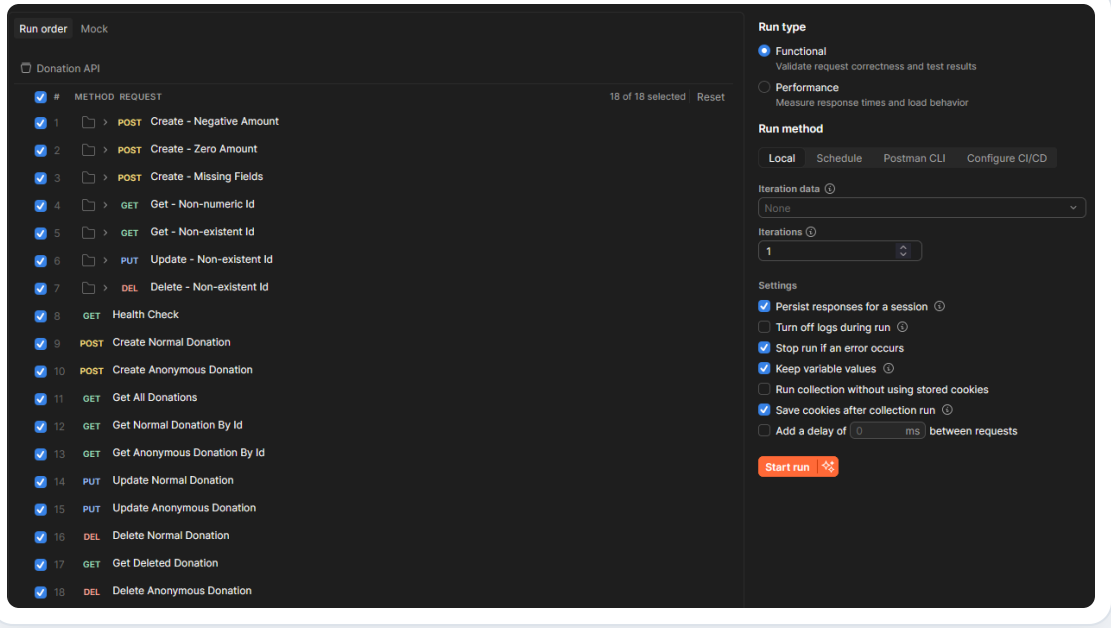
Validation & errors 7 requests

Scenario	Expected
Negative amount	400 · Amount error
Zero amount	400 · Amount error
Missing required fields	400 · Amount, IsAnonymous
Non-numeric id (`abc`)	400
Get non-existent id	404 · Donation not found
Update non-existent id	404
Delete non-existent id	404

Errors return clear ProblemDetails



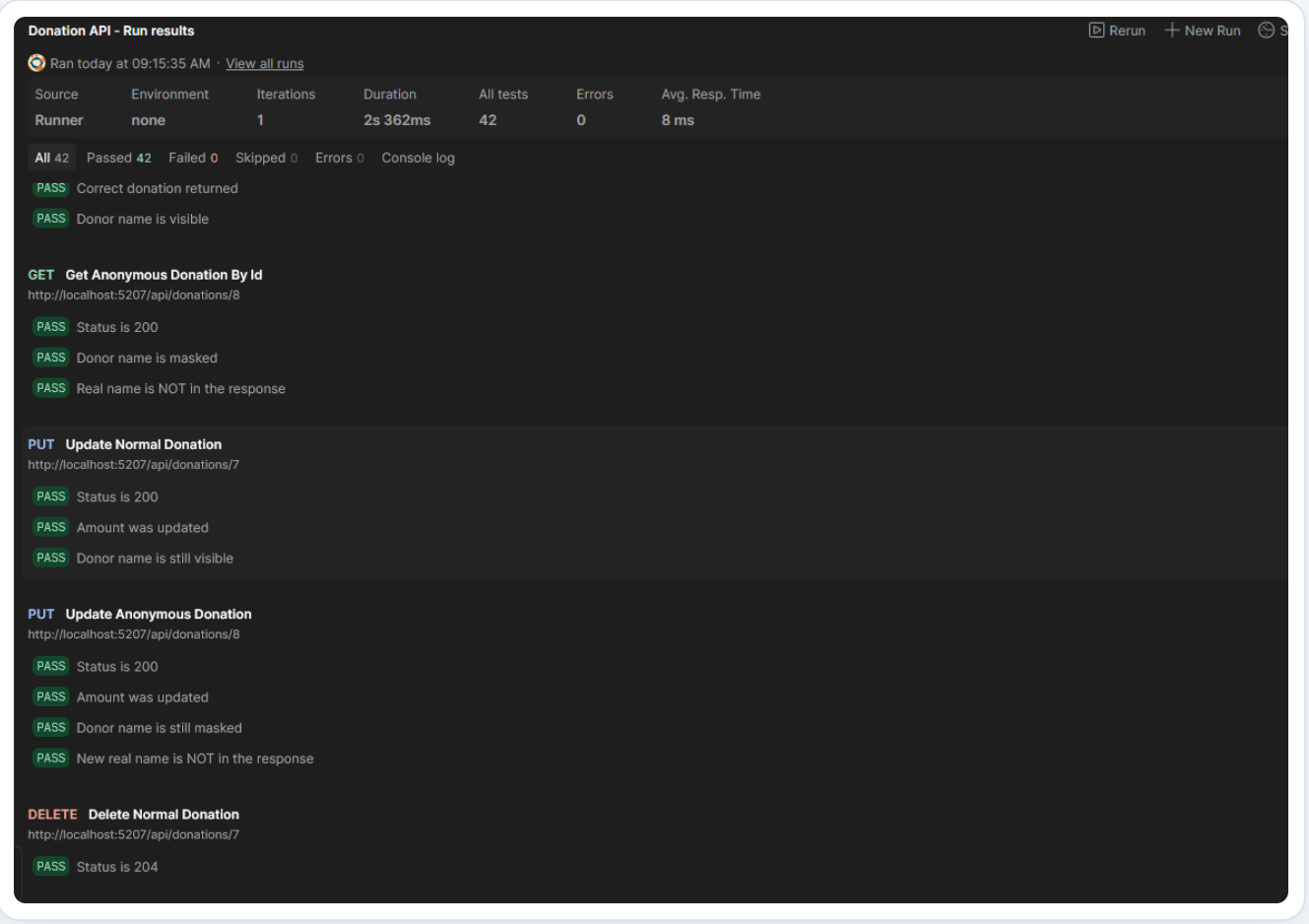
DELETE on id 999999 → 404 with `"title": "Donation not found"`, tests 2/2.



Collection Runner: 18 of 18 requests selected. Create runs before Read/Update, and Delete runs last, so the suite cleans up after itself.

RESULTS

42 tests · 42 passed · 0 failed



42
Passed

0
Failed

2.4s
Whole run

8 ms
Avg. response

The run covers every CRUD operation, the health check, the anonymity rule and all validation and error scenarios from the task.

DELIVERABLES CHECKLIST

Everything the task asked for

- **Complete .NET backend solution**: ASP.NET Core Web API, .NET 10
- **Database implementation**: EF Core migration `InitialCreate`
- **Working CRUD APIs + Health endpoint**: 6 endpoints
- **Swagger / OpenAPI**: models and status codes documented
- **Exported Postman collection**: `Postman/Donation API.postman_collection.json`
- **README.md**: setup, DB config, run, Swagger, Postman, design decisions

Scope respected

- **Backend only** no UI, Razor or HTML
- **No authentication** all endpoints public, as required
- **Anonymity** real names never returned

Run it: `dotnet ef database update` → `dotnet run --launch-profile http` → open `/swagger` or run the Postman collection.